INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2025

COURSE: [F25] Unix-based Internet Applications

CIA LAB 1 Report

On

Booting

Student Name: HASIB AL TAHSIN

Application ID: 187623

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 15.11.2025

Task 1 - PXE Installation

1. PXE Server Setup

1.1. Create the first virtual machine using VirtualBox and isolate a private network on your workstation: do not pollute our shared network with your own DHCP service. There are several network settings offered by VirtualBox, choose the right one accordingly and Install your PXE server there. Hint: use two network adapters.

1.2. Hint: you need to set up some of the services such as DHCP, TFTP, HTTP, NFS, and some boot loader (e.g. PXELINUX or GRUB) based on your approach. Also, take care of UEFI and BIOS based approaches. Hint: understand BIOS based approach but implement UEFI approach.

1.2.1. Write about each service's role in the PXE environment.

1.2.2. Your PXE server should serve the operating system of your choosing.

1.3. Question: why not run your DHCP service on the SNE network directly?

2. PXE Client Setup

2.1. Create the second virtual machine using VirtualBox in order to test the PXE service. 2.2. Change the boot order

2.3. Show that your PXE client takes the IP.

2.4. Boot and install a new system with it and show the proof in the report.

Task 2 - Questions to answer

1. Briefly explain UEFI with secure boot enabled, UEFI without secure boot, and BIOS PXE booting approaches.

1.1. How do they work? Explain with a simple diagram.

2. What is a GPT?

2.1. What is its general layout? Explain each element.

2.2. What is the role of a partition table?

3. What is gdisk?

3.1. How does it work?

3.2. What can you do with it?

3.3. Provide a simple practice.

4. What is a Protective MBR and why is it in the GPT?

Task 3 - Partitions

1. Verify the GPT schema of your Ubuntu machine.

2. Use the dd utility to dump the Protective MBR and GPT into a file in your home directory. The dump should contain up to first partition entry (Inclusive). Note: upload the dump file to your moodle.

3. Load the dump file into a hex dump utility (e.g. 010 editor) to look at the raw data in the file.

4. Understand and fully annotate the Protective MBR, GPT header and first partition entry in the report. This means you must describe the purpose of every field, and translate all fields that have a numerical value into human-readable, decimal format.

4.1. At what byte index from the start of the disk do the partition table entries start?

4.2. At what byte index would the partition table start if your server had a so-called “4K native” (4Kn) disk?

5. Name two differences between primary and logical partitions in an MBR partitioning scheme.

Task 1 (References [1-6])

1. PXE Server Setup

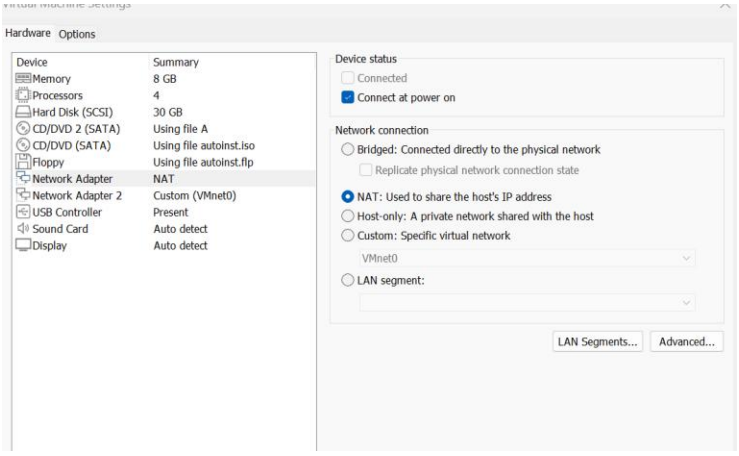
1.1. Creating the first virtual machine, installing a private network and PXE server

Creating a VM and isolating the network in VirtualBox:

We first created a new VM in VirtualBox, which will act as a PXE server. We used two network adapters in this VM:

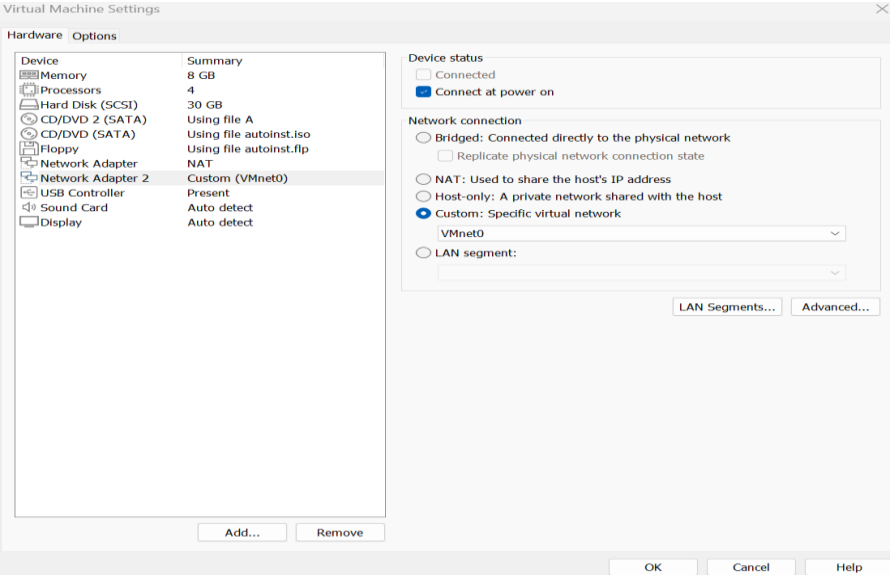
1. **Adapter 1:**

I put it on NAT, so that the PXE server can download packages using the internet.



2. **Adapter 2:**

I have set it up as a Host-only network, where my own DHCP server will run. This network is a separate “Custom: Specific virtual network - VMnet()”, so that it does not affect the SNE/shared network. So DHCP traffic is limited to this Internal network only, the external network (SNE) is not “polluted” at all.



PXE Server OS Installation

We have installed Ubuntu Server (which acts as a PXE server) on this VM. After the OS (Version: ubuntu-24.04.3-deskto) installation is complete, we have installed the services required for PXE:

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt-get -y install tftpd-hpa apache2 dnsmasq
[sudo] password for hasib:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tftpd-hpa is already the newest version (5.2+20150808-1.4build1).
apache2 is already the newest version (2.4.58-1ubuntu8.8).
dnsmasq is already the newest version (2.90-2ubuntu0.1).
0 upgraded, 0 newly installed, 0 to remove and 169 not upgraded.
```

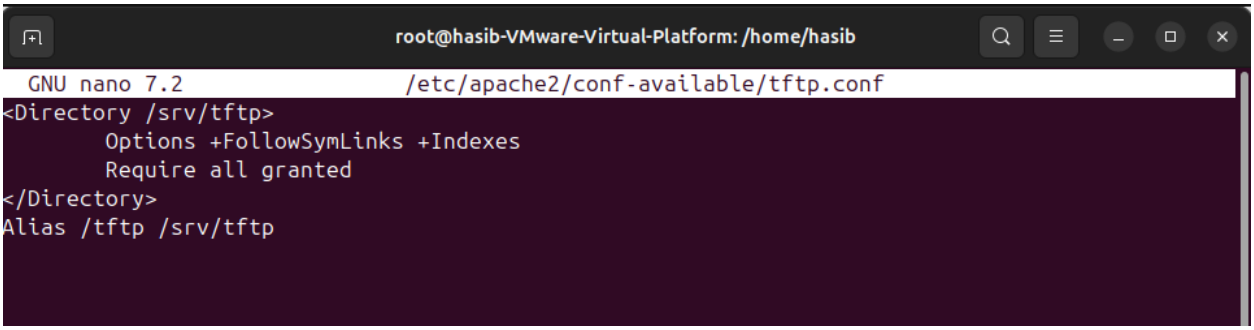
tftpd-hpa which acts as a TFTP server and apache2 which acts as an HTTP web server. dnsmasq which acts as a DHCP, PXE support and TFTP reference.

Serving TFTP folders with Apache

I have configured Apache to serve the /srv/tftp folder over HTTP:

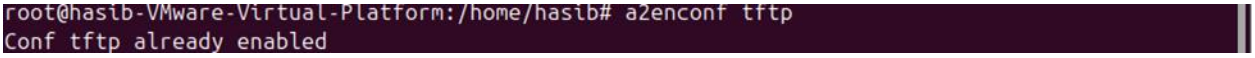
```
hasib@hasib-VMware-Virtual-Platform:~$ sudo su
root@hasib-VMware-Virtual-Platform:/home/hasib# nano /etc/apache2/conf-available/tftp.conf
```

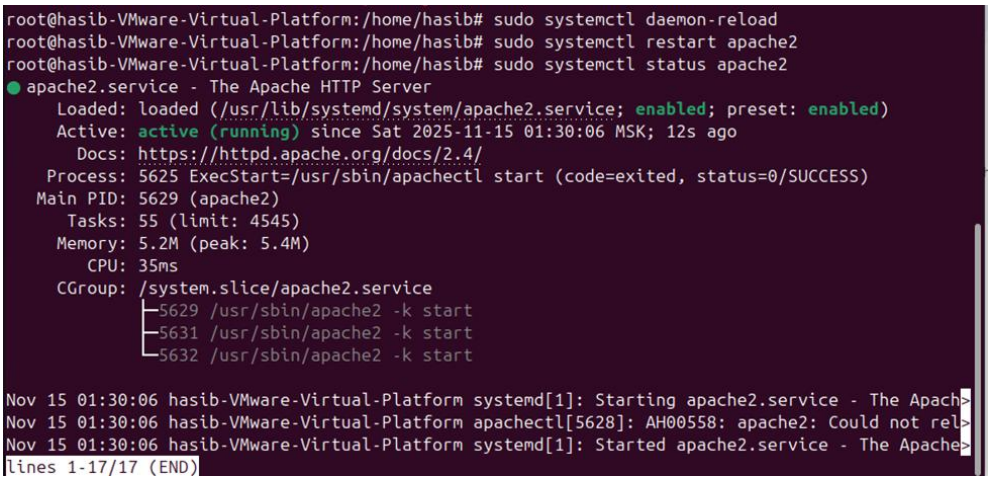
The config inside tftp.conf:



This configuration gives Apache public access to the /srv/tftp directory, lists files, allows symbolic links to be followed, and points the /tftp URL to that directory.

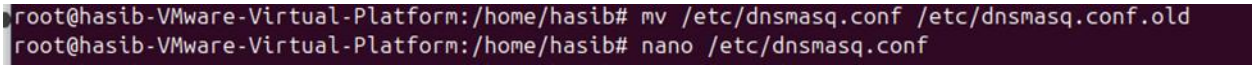
Then I enabled the config and restarted Apache:



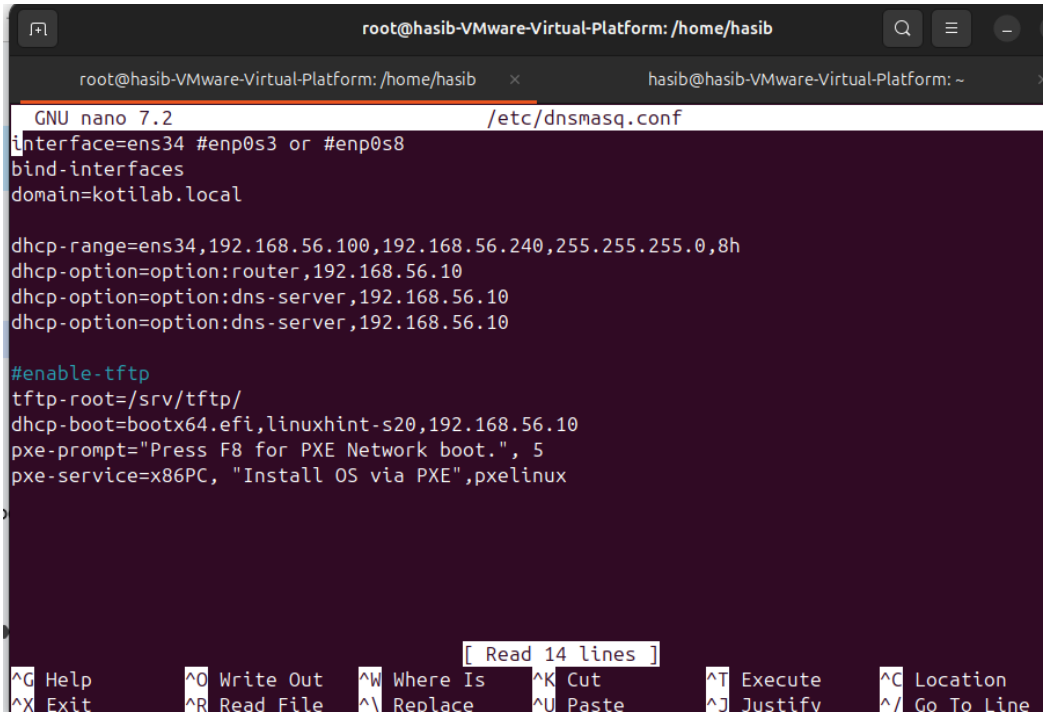


DHCP+PXE setup with dnsmasq

Backed up the old config and created a new /etc/dnsmasq.conf:



The old configuration file is saved and a new dnsmasq.conf is created. First, the previous file is saved as a backup with the name .old using the mv command, then the new dnsmasq.conf is opened for editing with nano. Inside is the DHCP config for PXE, which is added in the figure below:



In this configuration, dnsmasq has been bound to a specific interface and the DHCP service has been enabled, where the IP range, subnet, router and DNS address have been defined. EFI files, TFTP root directory and boot menu instructions have been added for PXE boot, so that the OS can be installed from the network. Finally, the dnsmasq service has been restarted and the status command has been used to verify whether the service is working properly.

EFI file ready for Secure Boot/UEFI

- Downloaded the required .deb file by going to /tmp and extracted the EFI file from there
- shim.signed to bootx64.efi to /srv/tftp/bootx64.efi
- grub-efi-amd64-signed to grubx64.efi to /srv/tftp/grubx64.efi
- grub-common to unicode.pf2 to /srv/tftp/unicode.pf2

The required .deb packages were downloaded by going to the /tmp directory and the EFI and required boot files were extracted from there. First, the bootx64.efi file was extracted from the shim.signed package and copied to /srv/tftp/bootx64.efi. Then, grubx64.efi was extracted from the grub-efi-amd64-signed package and placed in /srv/tftp/grubx64.efi. Finally, the font file unicode.pf2 used for booting was collected from the grub-common package and saved in /srv/tftp/unicode.pf2. In this way, all the EFI files required for PXE boot have been prepared.

```
root@hasib-VMware-Virtual-Platform:/home/hasib# cd /tmp
```

```
root@hasib-VMware-Virtual-Platform:/tmp# apt-get download shim.signed -y
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 shim-signed amd64 1.58+15.8-0ubuntu1 [665 kB]
Fetched 665 kB in 1s (467 kB/s)
```

```
root@hasib-VMware-Virtual-Platform:/tmp# dpkg-deb --fsys-tarfile /tmp/shim-signed*.deb \
  | tar -xOf - ./usr/lib/shim/shimx64.efi.signed.latest \
  > /srv/tftp/bootx64.efi
```

```
root@hasib-VMware-Virtual-Platform:/tmp# apt download grub-efi-amd64-signed
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 grub-efi-amd64-signed amd64 1.202.5+2.12-1u
buntu7.3 [1,393 kB]
Fetched 1,393 kB in 1s (1,685 kB/s)
```

```
root@hasib-VMware-Virtual-Platform:/tmp# dpkg-deb --fsys-tarfile /tmp/grub-efi-amd64-signed*deb | tar x ./u
sr/lib/grub/x86_64-efi-signed/grubnetx64.efi.signed -O >  /srv/tftp/grubx64.efi
root@hasib-VMware-Virtual-Platform:/tmp#
```

```
root@hasib-VMware-Virtual-Platform:/tmp# apt download grub-common
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 grub-common amd64 2.12-1ubuntu7.3 [2,120 kB
]
Fetched 2,120 kB in 2s (1,245 kB/s)
root@hasib-VMware-Virtual-Platform:/tmp#
```

```
root@hasib-VMware-Virtual-Platform:/tmp# dpkg-deb --fsys-tarfile grub-common*.deb \
  | tar -xOf - ./usr/share/grub/unicode.pf2 \
  > /srv/tftp/unicode.pf2
```

Ubuntu kernel/initrd and ISO ready

Mounted ISO file /srv/tftp/ubuntu-20.04-live-server-amd64.iso :

```
root@hasib-VMware-Virtual-Platform:/home/hasib# ls -lh /srv/tftp/
total 2.0G
-rw-r--r-- 1 tftp tftp    6 Nov 15 03:14 test.txt
-rw-r--r-- 1 root root    0 Nov 15 03:25 ubuntu-20.04-live-server-amd64.iso
-rw-r--r-- 1 root root 2.0G Sep 11  2024 ubuntu-22.04.5-live-server-amd64.iso
root@hasib-VMware-Virtual-Platform:/home/hasib# sudo mount -o loop /srv/tftp/ubuntu-22.04.5-live-server-amd
64.iso /mnt/
mount: /mnt: WARNING: source write-protected, mounted read-only.
root@hasib-VMware-Virtual-Platform:/home/hasib#
```

Then create directories like /srv/tftp/focal:

```
root@hasib-VMware-Virtual-Platform:/home/hasib# ls -lh /mnt
total 32K
dr-xr-xr-x 1 root root 2.0K Aug 13  2024 boot
-r--r--r-- 1 root root 2.0K Sep 11  2024 boot.catalog
dr-xr-xr-x 1 root root 6.0K Sep 11  2024 casper
dr-xr-xr-x 1 root root 2.0K Sep 11  2024 dists
dr-xr-xr-x 1 root root 2.0K Aug 13  2024 EFI
dr-xr-xr-x 1 root root 2.0K Sep 11  2024 install
-r--r--r-- 1 root root  14K Sep 11  2024 md5sum.txt
dr-xr-xr-x 1 root root 2.0K Sep 11  2024 pool
lr-xr-xr-x 1 root root    1 Sep 11  2024 ubuntu -> .
```

Then copying vmlinuz and initrd from ISO:

```
root@hasib-VMware-Virtual-Platform:/home/hasib# cp /mnt/casper/vmlinuz /srv/tftp/blade/
root@hasib-VMware-Virtual-Platform:/home/hasib# cp /mnt/casper/initrd /srv/tftp/blade/
```

GRUB Config (UEFI PXE Boot)

GRUB config file /srv/tftp/grub/grub.cfg created:

The /srv/tftp/grub/grub.cfg file has been created to enable PXE based automated OS installation through GRUB bootloader. The new GRUB config file has been edited using nano. Here, first, we have set the default menu entry with default=autoinstall, so that if no selection is made from the boot menu, the specific installer is automatically selected. Timeout=30 has been set to wait for 30 seconds and timeout_style=menu has been set to display a menu-based countdown.

Then, a menuentry has been created, named "22 server Installer - automated" and its ID is autoinstall. When this entry is selected, GRUB first loads the /jammy-server/vmlinuz kernel, gets an IP from the network using DHCP and starts the automated installation by loading the ISO file from the specified HTTP address. The cloud-init directory is obtained from the server using the autoinstall and nocloud-net parameters. Then, the initrd file is loaded with the message "Loading Ram Disk..." A complete PXE-based unattended installation environment has been prepared.

```
root@hasib-VMware-Virtual-Platform:/tmp# nano /srv/tftp/grub/grub.cfg
```

```
  GNU nano 7.2                                /srv/tftp/grub/grub.cfg
default=autoinstall
timeout=30
timeout_style=menu

menuentry "22.04.5 Server Installer - automated" --id=autoinstall {
    linux /blade/vmlinuz ip=dhcp \
    url=http://192.168.56.10/tftp/blade/ubuntu-22.04.5-live-server-amd64.iso \
    autoinstall ds='nocloud-net;s=http://192.168.56.10/tftp/blade/' \
    cloud-config-url=/dev/null root=/dev/ram0

    echo "Loading Ram Disk..."
    initrd /blade/initrd
}
```

cloud-init (autoinstall) config

meta-data and user-data:

Two important files of cloud-init are created for PXE based automatic Ubuntu installation, meta-data and user-data. First, an empty meta-data file is created with the touch /srv/tftp/focal/meta-data command, because the meta-data file is mandatory in Ubuntu autoinstall mode, although it usually does not require any content. Then, the mkpasswd --method=sha-512 ubuntu command is used to

securely generate the system's default user password. This command generates a SHA-512 hashed password, which is later used in the user-data file for secure login.

```
root@hasib-VMware-Virtual-Platform:/tmp# touch /srv/tftp/blade/meta-data
root@hasib-VMware-Virtual-Platform:/tmp#
```

```
root@hasib-VMware-Virtual-Platform:/tmp# mkpasswd --method=sha-512 ubuntu
$6$jhkZbOQrih0osZDh$M2Pwn2reYouicSfuaRX69SFrqyI6i7WzRNJ92e/nlQmmmipTQRH0NSr60E94FUmZ1FnAKB49BHnVrfuVWUoZh/
```

Then the user-data file is edited with the command nano /srv/tftp/focal/user-data. This file contains all the instructions for the automated installation - such as user creation, hostname, timezone, network configuration, disk partitioning, package installation, SSH access, etc. The installation is completely automated through the user-data file and no human input is required. In this way, a complete unattended setup has been prepared using cloud-init.

```
root@hasib-VMware-Virtual-Platform:/tmp# nano /srv/tftp/blade/user-data
```

```
 GNU nano 7.2                          /srv/tftp/blade/user-data *
#cloud-config
autoinstall:
  version: 1
  identity:
    hostname: blade
    username: hasib
    password: $6$jhkZbOQrih0osZDh$M2Pwn2reYouicSfuaRX69SFrqyI6i7WzRNJ92e/nlQmmmipTQRH0NSr60E94FUmZ1FnAKB49>
```

user-data:

This cloud-init-based #cloud-config file is used to configure Ubuntu's autoinstall process. The autoinstall: block specifies the system installation settings. The identity section specifies the default user created after installation. The hostname: blade specifies the system hostname. The username: hasib, specifies the default username, and the password: field specifies a secure hash password generated using the SHA-512 algorithm, which is used for login. version: 1 specifies the schema version of the cloud-init configuration.

```
 GNU nano 7.2                                /srv/tftp/blade/user-data *
#cloud-config
autoinstall:
  version: 1
  identity:
    hostname: blade
    username: hasib
    password: $6$jhkZbOQrih0osZDh$M2Pwn2reYouicSfuaRX69SFrqyI6i7WzRNJ92e/nlQmmmipTQRH0NSr60E94FUmZ1FnAKB49>
```

Permission and Ownership

• Setting the owner and permission of the /srv/tftp directory:

In this step, the ownership of the `/srv/tftp` directory is set to the tftp user name and full read, write and execute permissions are given with `chmod 777`, which allows the PXE boot files to be accessed properly.

```
root@hasib-VMware-Virtual-Platform:/tmp# chown tftp: -R /srv/tftp/

root@hasib-VMware-Virtual-Platform:/tmp# chmod 777 -R /srv/tftp/
```

1.2. Useful services (DHCP, TFTP, HTTP, NFS, boot loader, UEFI/BIOS)

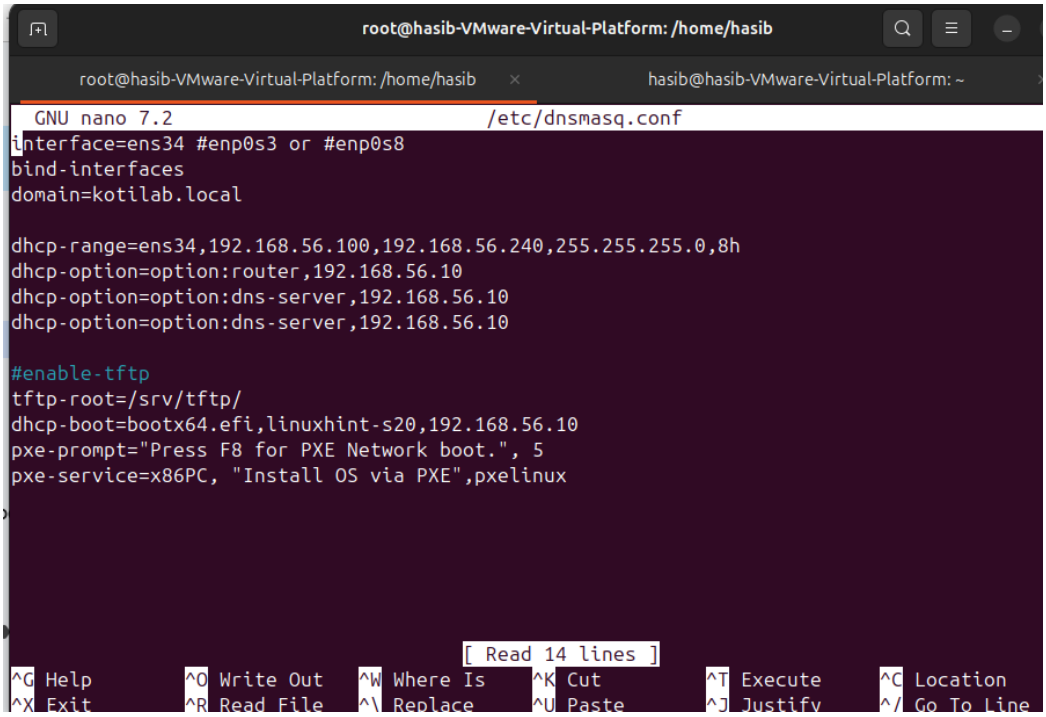
1.2.1. Role of each service in PXE Environment

The service-by-service introduction is given below:

- **DHCP Service (dnsmasq)**

In a network-based installation infrastructure, dnsmasq acts as an important component providing an integrated DHCP, DNS and TFTP service. One of its main responsibilities is to dynamically distribute IP addresses to clients. In the corresponding configuration, the IP range and lease time are defined through a specific interface using the dhcp-range=ens38,192.168.10.100,192.168.10.240,255.255.255.0,8h directive. As a result, a valid IP is automatically obtained when a new PXE client joins the network. To define the basic infrastructure for network communication, dhcp-option=option:router,192.168.10.1 and dhcp-option=option:dns-server,192.168.10.1 have been added, which specify the default gateway and DNS server addresses for the client. This enables the client to perform network routing and name resolution correctly. Another role of dnsmasq in a PXE-bootable environment is to inform the client of the location of the TFTP server and boot files. The dhcp-boot=bootx64.efi,linuxhint-s20,192.168.10.100 directive provides the client with the path to the EFI-based bootloader and

specifies the IP of the target server. To improve PXE-interaction, pxe-prompt and pxe-service are used, which display boot menu prompts and help the client locate available PXE.



- **TFTP Service (tftpd-hpa)**

The PXE client first gets the TFTP server and boot file name from DHCP. Then it downloads small files like EFI loader, GRUB, kernel, initrd etc. via TFTP.

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt-get -y install tftpd-hpa apache2 dnsmasq
[sudo] password for hasib:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tftpd-hpa is already the newest version (5.2+20150808-1.4build1).
apache2 is already the newest version (2.4.58-1ubuntu8.8).
dnsmasq is already the newest version (2.90-2ubuntu0.1).
0 upgraded, 0 newly installed, 0 to remove and 169 not upgraded.
```

- **HTTP Service (apache2)**

Large files, such as the Ubuntu server ISO (ubuntu-22.04.3-live-server-amd64.iso) and cloud-init data (user-data, meta-data) are sent to the client over HTTP. Using HTTP generally improves the speed and reliability of PXE installations, and works without NFS.

```
root@hasib-VMware-Virtual-Platform:/home/hasib# sudo systemctl daemon-reload
root@hasib-VMware-Virtual-Platform:/home/hasib# sudo systemctl restart apache2
root@hasib-VMware-Virtual-Platform:/home/hasib# sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Sat 2025-11-15 01:30:06 MSK; 12s ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 5625 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 5629 (apache2)
      Tasks: 55 (limit: 4545)
     Memory: 5.2M (peak: 5.4M)
        CPU: 35ms
     CGroup: /system.slice/apache2.service
             ├─5629 /usr/sbin/apache2 -k start
             ├─5631 /usr/sbin/apache2 -k start
             └─5632 /usr/sbin/apache2 -k start

Nov 15 01:30:06 hasib-VMware-Virtual-Platform systemd[1]: Starting apache2.service - The Apach>
Nov 15 01:30:06 hasib-VMware-Virtual-Platform apachectl[5628]: AH00558: apache2: Could not rel>
Nov 15 01:30:06 hasib-VMware-Virtual-Platform systemd[1]: Started apache2.service - The Apache>
lines 1-17/17 (END)
```

- **Boot Loader (UEFI – shim + GRUB)**

Two important files are used to enable EFI-based booting in a PXE environment: /srv/tftp/bootx64.efi, which is the Secure Boot–compatible shim loader, and /srv/tftp/grubx64.efi, which is the EFI netboot loader for GRUB. /srv/tftp/grub/grub.cfg is used for GRUB configuration. During the boot process, UEFI first loads the shim (bootx64.efi), then starts the shim GRUB (grubx64.efi). GRUB then reads grub.cfg, displays the menu, and starts the PXE installation with the autoinstall parameter, loading the kernel (linux) and initrd according to the specified menuentry. The PXELINUX/syslinux bootloader could have been used, but we implemented the UEFI approach for the purpose.

```
root@hasib-VMware-Virtual-Platform:/tmp# nano /srv/tftp/grub/grub.cfg
```

```
  GNU nano 7.2                                /srv/tftp/grub/grub.cfg
default=autoinstall
timeout=30
timeout_style=menu

menuentry "22.04.5 Server Installer - automated" --id=autoinstall {
    linux /blade/vmlinuz ip=dhcp \
    url=http://192.168.56.10/tftp/blade/ubuntu-22.04.5-live-server-amd64.iso \
    autoinstall ds='nocloud-net;s=http://192.168.56.10/tftp/blade/' \
    cloud-config-url=/dev/null root=/dev/ram0

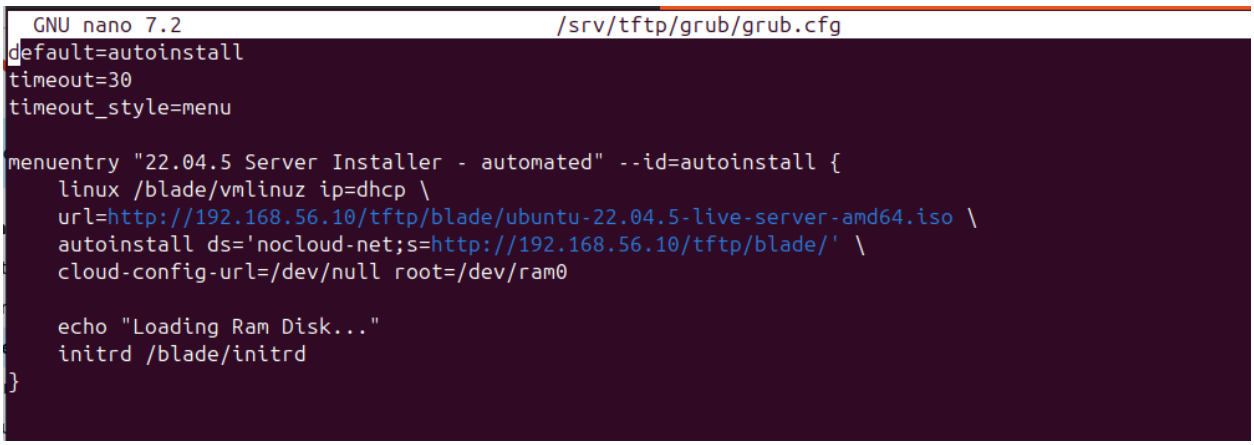
    echo "Loading Ram Disk..."
    initrd /blade/initrd
}
```

1.2.2. OS that the PXE server is serving:

According to the configuration, the PXE server is basically serving Ubuntu Server:

- **Ubuntu 22.04.3 (blade)**

Three components are used for the PXE-based automated installation - /srv/tftp/ubuntu-22.04.3-live-server-amd64.iso is the original Ubuntu ISO, which is referenced over the network to create the installer environment. /srv/tftp/focal/vmlinuz and /srv/tftp/focal/initrd are the kernel and initrd, which load GRUB to start PXE booting and accept autoinstall parameters. /srv/tftp/focal/meta-data and /srv/tftp/focal/user-data are used as the cloud-init configuration, where the meta-data file is for base identity information and the user-data file contains the full automated installation instructions, such as user, password, network, partition, and other settings.



- **Ubuntu 24.04 (hasib)**

Ubuntu Server (22.04.03/24.04 (Main Server PC OS)) was installed from PXE server in network boot and autoinstall mode. UEFI Boot Mode was used while installing Main Server Ubuntu OS (24.04).

1.3. Reasons why I should not run DHCP service directly on the SNE network:

Reasons:

- **Risk of DHCP conflict / IP conflict**

Shared network already has official DHCP server. If I run another DHCP server (dnsmasq) on our own, where some machines have official DHCP and some machines have our PXE DHCP. This can lead to IP conflict, wrong gateway, wrong DNS etc., which can cause big problems for others on the network.

- **Security and policy violations**

Running DHCP without permission on educational institutions/office production networks is generally strictly forbidden. Network admins control the IP plan of the entire network, where running unauthorized DHCP can even bring the network down.

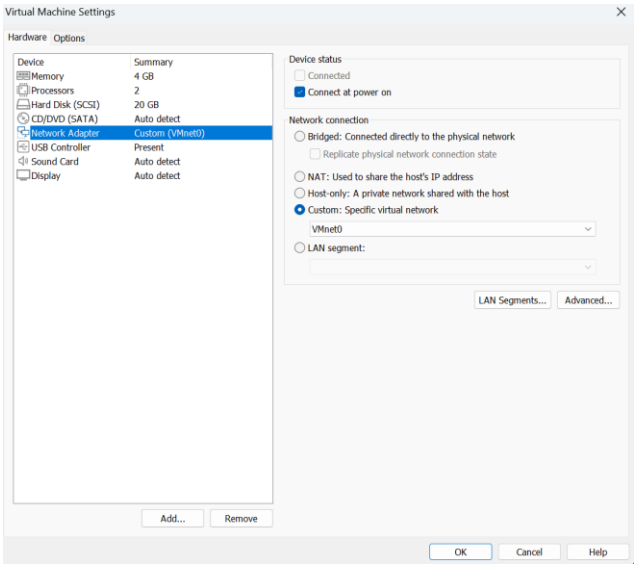
- **A separate private network is sufficient for labs/projects**

For the PXE lab, I only need VM-to-VM communication. So the second adapter is used in VMware's Internal Network/Host-only mode - this keeps DHCP limited to that virtual lab network and the real network is completely secure.

2. PXE Client Setup

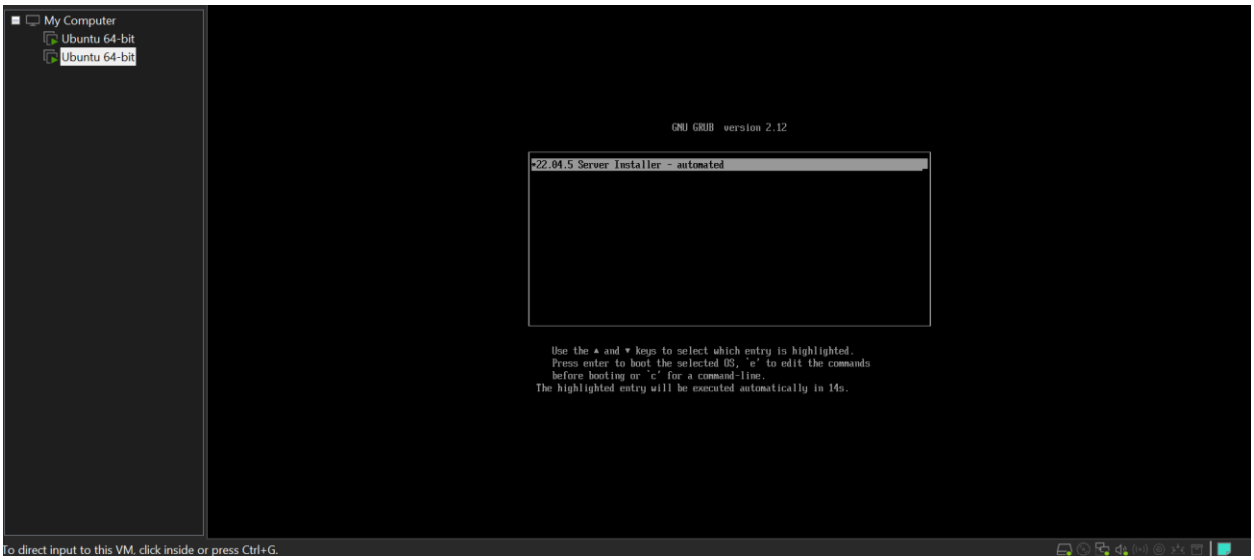
2.1. Creating a second virtual machine (PXE Client)

- Another VM is created in VirtualBox which acts as a PXE client.
- Network config of this 2nd VM.
- One of its network adapters is connected to the same Specific Virtual Network - VMnet(), where the 1st VM Server PC is connected.
- As a result, the PXE client VM directly accesses the dnsmasq/DHCP and TFTP of the PXE server.
- This VM can boot without having any OS installed beforehand; our goal is to install the OS from PXE.

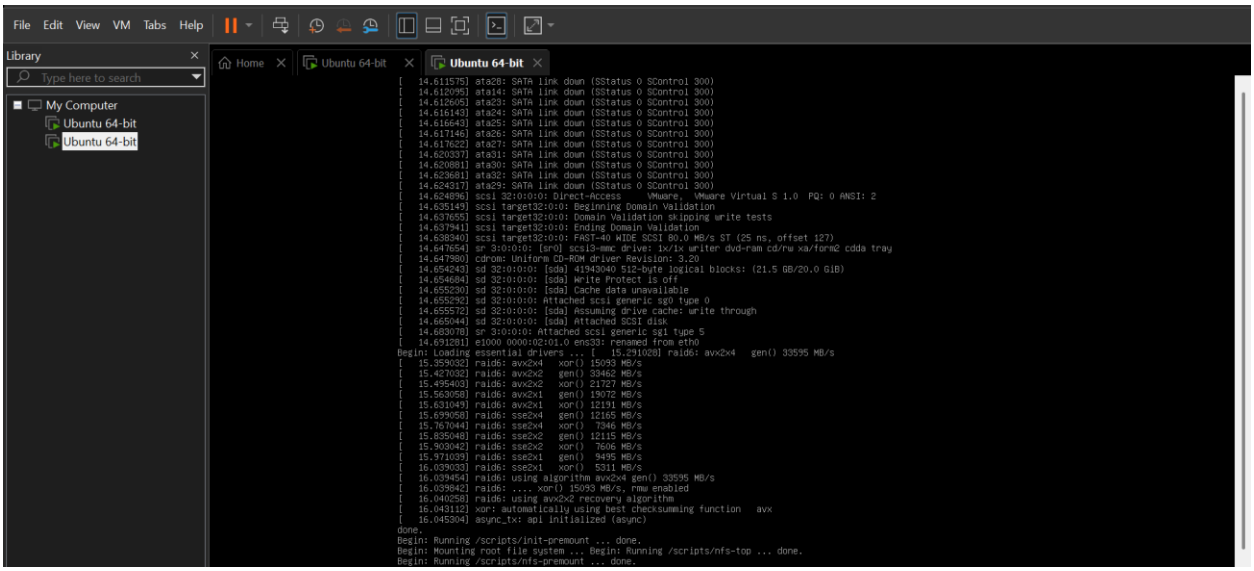


❖ The ubuntu boot at Second VM PC or Client PC

The installation page is shown from the second PC at Network boot



The installation runs correctly at Network Boot



The screen above shows the booting from the PXE client, although the screenshot mainly shows the boot process.

2.2. Changing the boot order (Network Boot Priority)

On the PXE client VM:

- Going to Settings Boot Order of VMware Workstation 17 Pro:

Network (or VMnet()) is not selected as the first boot device, because although VirtualBox has this facility, I did not get this setup in VMware Workstation 17 Pro. The setup of hard disk, optical etc. is kept as it is. Also, the first PC or server PC is booted in UEFI Mode. The VM machine boots network (PXE) from UEFI firmware and loads the bootx64.efi file from TFTP.

```
root@hasib-VMware-Virtual-Platform:/tmp# dpkg-deb --fsys-tarfile /tmp/shim-signed*.deb \
  | tar -xOf - ./usr/lib/shim/shimx64.efi.signed.latest \
  > /srv/tftp/bootx64.efi
```

```
root@hasib-VMware-Virtual-Platform:/tmp# apt download grub-efi-amd64-signed
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 grub-efi-amd64-signed amd64 1.202.5+2.12-1u
buntu7.3 [1,393 kB]
Fetched 1,393 kB in 1s (1,685 kB/s)
```

```
root@hasib-VMware-Virtual-Platform:/tmp# dpkg-deb --fsys-tarfile /tmp/grub-efi-amd64-signed*deb | tar x ./u
sr/lib/grub/x86_64-efi-signed/grubnetx64.efi.signed -O >  /srv/tftp/grubx64.efi
root@hasib-VMware-Virtual-Platform:/tmp#
```

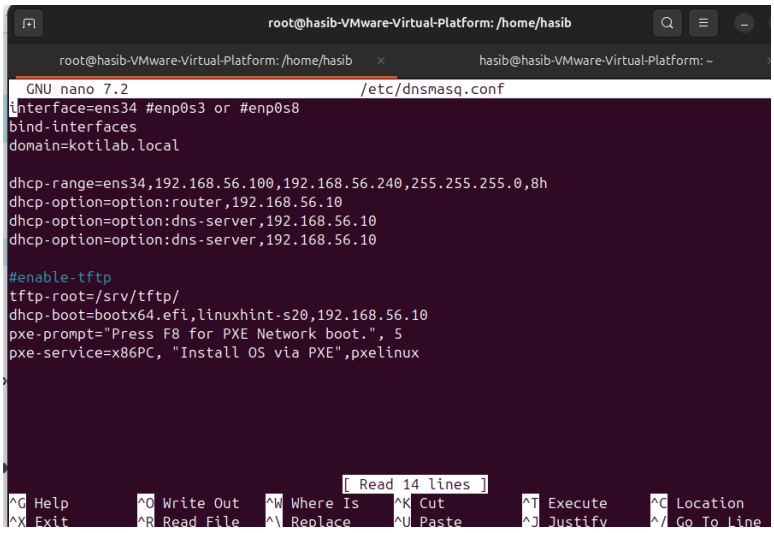
Due to not fixing the boot order, when the client VM starts, it first tries to boot other devices and then tries to boot the network, which sends traffic to our PXE server.

2.3. Proof: PXE client getting IP How to see if client got IP:

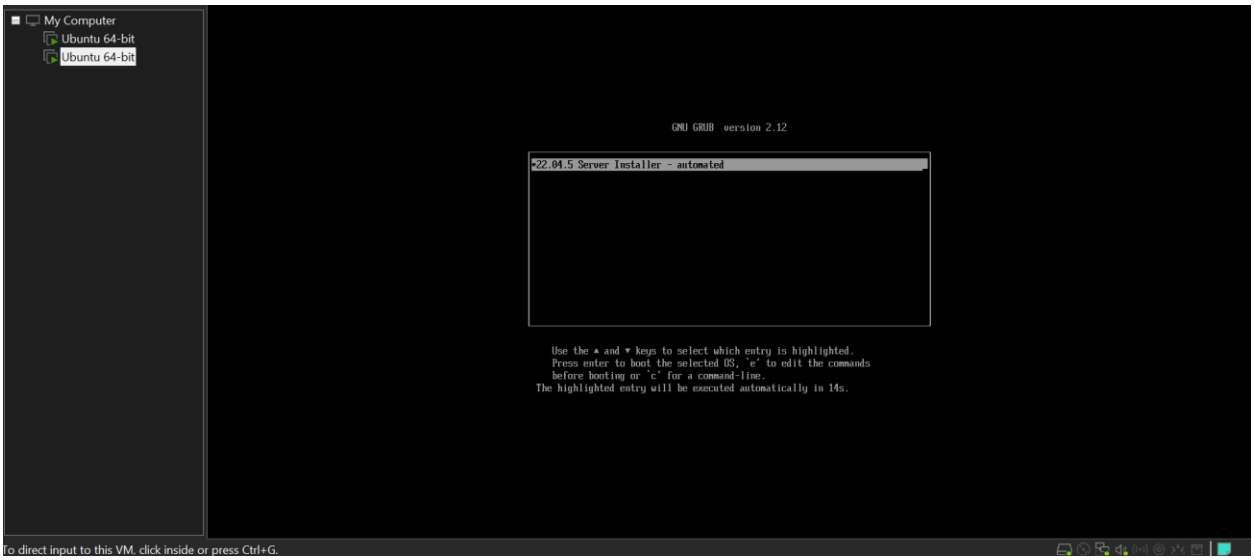
- **IP Range as per dnsmasq config**

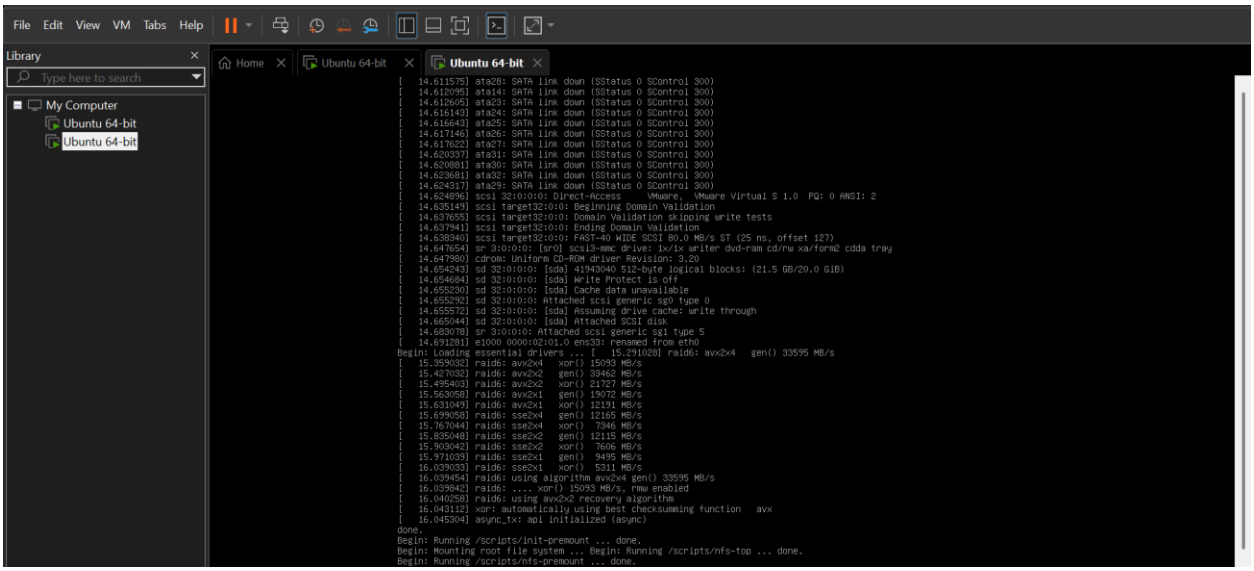
  In /etc/dnsmasq.conf:

  - dhcp-range=ens38,192.168.10.100,192.168.10.240,255.255.255.0,8h
  - When the 2nd VM PXE client boots from the network, dnsmasq gives it an IP between 192.168.10.100 and 192.168.10.240.



- **Client boot screen**
    - When booting the PXE client VM, it first displays a message, then bootx64.efi is loaded via UEFI PXE.
    - The figure below shows the PXE client has arrived at the Ubuntu boot screen over the network - this means DHCP has successfully provided IP and TFTP information.





- **Viewing status/log from server**

  On the PXE server side:

    - systemctl status dnsmasq
    - The log here (status) shows that a new DHCP lease has been issued (active).

```
root@hasib-VMware-Virtual-Platform:/home/hasib# sudo systemctl status dnsmasq
● dnsmasq.service - dnsmasq - A lightweight DHCP and caching DNS server
     Loaded: loaded (/usr/lib/systemd/system/dnsmasq.service; enabled; preset: enabled)
     Active: active (running) since Sat 2025-11-15 02:04:53 MSK; 7s ago
    Process: 7162 ExecStartPre=/usr/share/dnsmasq/systemd-helper checkconfig (code=exited, sta>
    Process: 7168 ExecStart=/usr/share/dnsmasq/systemd-helper exec (code=exited, status=0/SUCC>
    Process: 7174 ExecStartPost=/usr/share/dnsmasq/systemd-helper start-resolvconf (code=exite>
   Main PID: 7173 (dnsmasq)
      Tasks: 1 (limit: 4545)
     Memory: 1.3M (peak: 2.9M)
        CPU: 48ms
     CGroup: /system.slice/dnsmasq.service
             └─7173 /usr/sbin/dnsmasq -x /run/dnsmasq/dnsmasq.pid -u dnsmasq -r /run/dnsmasq/r>

Nov 15 02:04:53 hasib-VMware-Virtual-Platform systemd[1]: Starting dnsmasq.service - dnsmasq -
Nov 15 02:04:53 hasib-VMware-Virtual-Platform dnsmasq[7173]: started, version 2.90 cachesize 1>
Nov 15 02:04:53 hasib-VMware-Virtual-Platform dnsmasq[7173]: compile time options: IPv6 GNU-ge>
Nov 15 02:04:53 hasib-VMware-Virtual-Platform dnsmasq-dhcp[7173]: DHCP, IP range 192.168.56.10>
Nov 15 02:04:53 hasib-VMware-Virtual-Platform dnsmasq-dhcp[7173]: DHCP, sockets bound exclusiv>
Nov 15 02:04:53 hasib-VMware-Virtual-Platform dnsmasq[7173]: read /etc/hosts - 8 names
Nov 15 02:04:53 hasib-VMware-Virtual-Platform resolvconf[7181]: Dropped protocol specifier '.d>
Nov 15 02:04:53 hasib-VMware-Virtual-Platform resolvconf[7181]: Failed to set DNS configuratio>
Nov 15 02:04:53 hasib-VMware-Virtual-Platform systemd[1]: Started dnsmasq.service - dnsmasq - >
lines 1-22/22 (END)
```

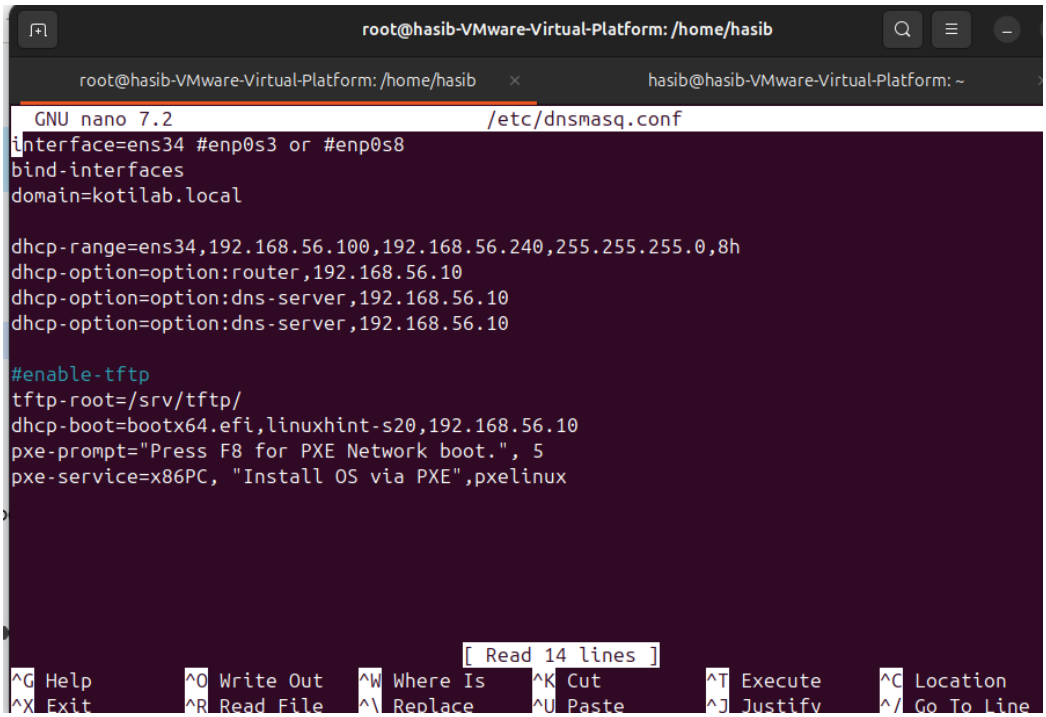
2.4. Booting and installing a new system with PXE:

Boot process (UEFI PXE + GRUB + autoinstall)

When starting the PXE client VM:

- **UEFI firmware to DHCP**

The client sends a DHCP request to the network and receives the IP and bootx64.efi information from dnsmasq.



- **Loading EFI Loader with TFTP**

The client downloaded /srv/tftp/bootx64.efi (shim) via TFTP.

```
root@hasib-VMware-Virtual-Platform:/tmp# dpkg-deb --fsys-tarfile /tmp/shim-signed*.deb \
  | tar -xOf - ./usr/lib/shim/shimx64.efi.signed.latest \
  > /srv/tftp/bootx64.efi
```

GRUB loads EFI from shim.

```
root@hasib-VMware-Virtual-Platform:/tmp# dpkg-deb --fsys-tarfile /tmp/grub-efi-amd64-signed*deb | tar x ./u
sr/lib/grub/x86_64-efi-signed/grubnetx64.efi.signed -O >  /srv/tftp/grubx64.efi
root@hasib-VMware-Virtual-Platform:/tmp#
```

GRUB menu and kernel loading

GRUB reads the /srv/tftp/grub/grub.cfg file during boot and selects the "22 server Installer - automated" menu there by default. When this entry is enabled, GRUB runs the linux /blade/vmlinuz command to load the kernel from the network, which includes the DHCP IP, the HTTP URL of the ISO, and the cloud-init instructions for autoinstall. The initrd /blade-server/initrd is then loaded. As a result, both the kernel and initrd are loaded from TFTP/HTTP into RAM and begin the automated installation.

```
root@hasib-VMware-Virtual-Platform:/tmp# nano /srv/tftp/grub/grub.cfg
```

```
  GNU nano 7.2                              /srv/tftp/grub/grub.cfg
default=autoinstall
timeout=30
timeout_style=menu

menuentry "22.04.5 Server Installer - automated" --id=autoinstall {
    linux /blade/vmlinuz ip=dhcp \
    url=http://192.168.56.10/tftp/blade/ubuntu-22.04.5-live-server-amd64.iso \
    autoinstall ds='nocloud-net;s=http://192.168.56.10/tftp/blade/' \
    cloud-config-url=/dev/null root=/dev/ram0

    echo "Loading Ram Disk..."
    initrd /blade/initrd
}
```

- **Autoinstall: Installation without any input**

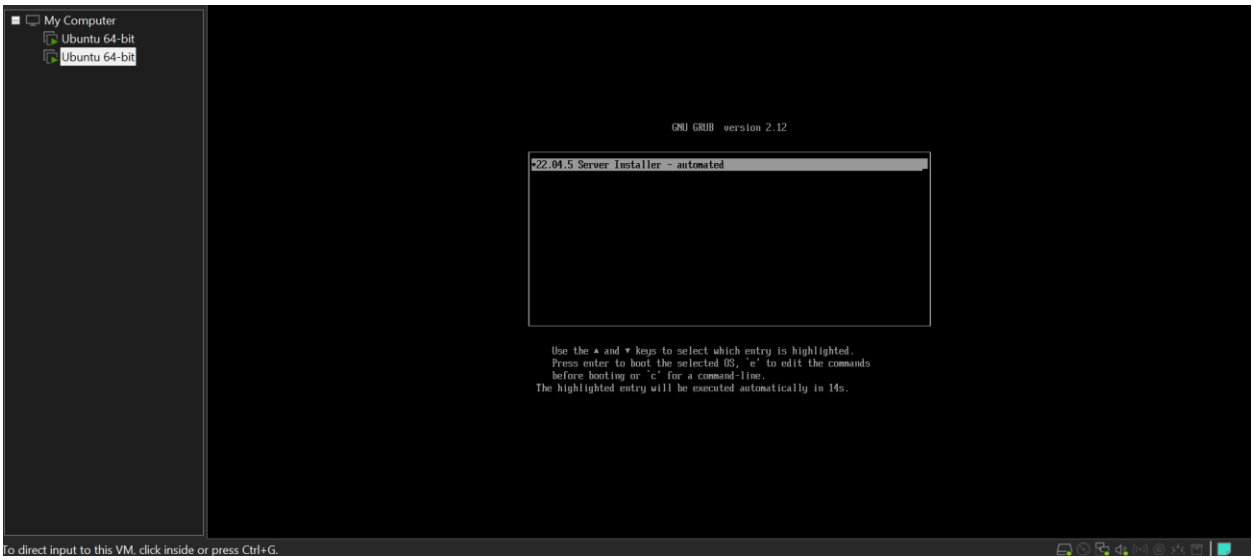
The user-data is given as:

- hostname: blade
- username: hasib
- password: (SHA-512 hashed)

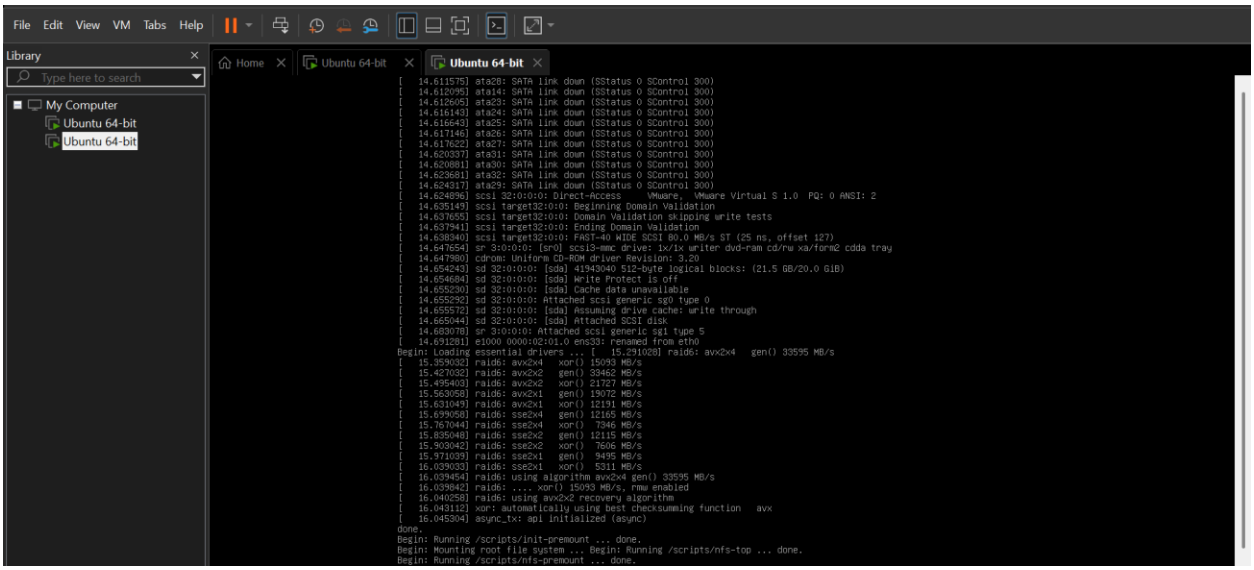
So on the Ubuntu client PC, the installer automatically created the user and installed the base system without asking any questions.

```
  GNU nano 7.2                          /srv/tftp/blade/user-data *
#cloud-config
autoinstall:
  version: 1
  identity:
    hostname: blade
    username: hasib
    password: $6$jhkZbOQrih0osZDh$M2Pwn2reYouicSfuaRX69SFrqyI6i7WzRNJ92e/nlQmmmipTQRH0NSr60E94FUmZ1FnAKB49>
```

The figure below shows the Ubuntu boot screen over the network of the second VM (client PC) - meaning it has successfully booted from the main server.



The figure below shows the installation running properly via network boot, where the installer has completely installed the OS from the network source. Boot and install a new Ubuntu system with it.

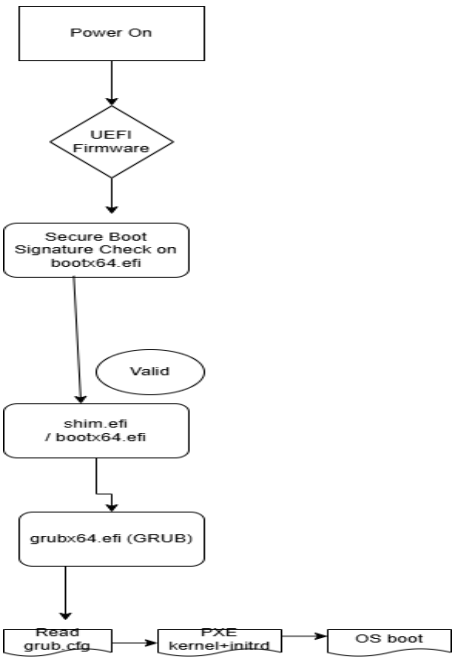


Task 2 (References [7-16])

1. UEFI & BIOS PXE Booting Approaches

1. UEFI with Secure Boot Enabled

UEFI Secure Boot is a security system that allows only trusted (signed) bootloaders and OS to be booted. The firmware verifies the signature of the EFI file before booting. When the machine is booted, the UEFI firmware first initializes the hardware and activates the Secure Boot policy. The firmware then verifies the signature of the bootx64.efi file received from PXE using the authorized signing key in its Trust Store. If the signature is correct, the EFI loader (shim) is started. The shim securely loads the GRUB EFI (grubx64.efi) and GRUB reads its configuration file grub.cfg to determine the next steps. GRUB then uses the PXE network to download the kernel (vmlinuz) and initrd into RAM, from which the main boot process begins.

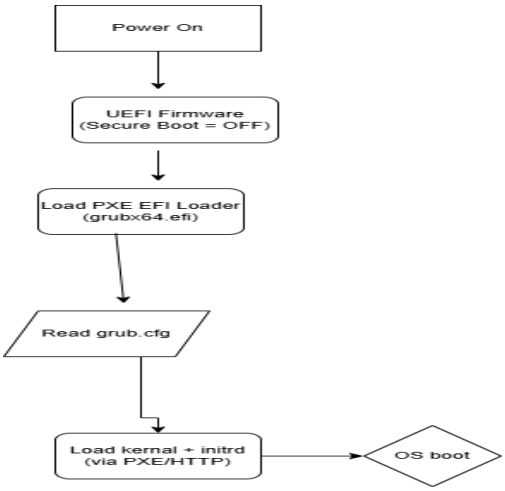


UEFI without Secure Boot

UEFI firmware is present but Secure Boot is disabled. The firmware does not check the signature of the EFI file; any valid EFI bootloader can be launched.

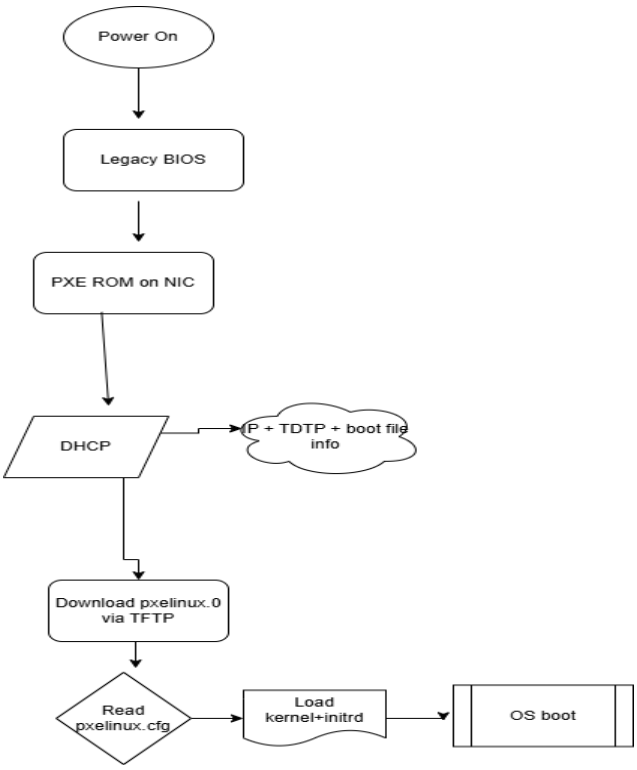
Steps to work (in case of PXE):

- When the machine is first turned on, the UEFI firmware starts.
- Since Secure Boot is turned off, the firmware does not check any signatures.
- grubx64.efi or another EFI loader is loaded directly from the network PXE.
- GRUB config is read from the EFI loader, kernel and initrd are loaded and OS boot begins.



BIOS PXE Booting

In BIOS-based PXE boot, the old Legacy BIOS and the PXE ROM (code contained in the Network Interface Card) work together. There is no EFI file, where PXELINUX/syslinux or similar bootloaders are used. When the machine is first started, the Legacy BIOS first initializes the hardware and selects Network Boot (PXE) according to the specified boot order. Then the NIC's PXE ROM broadcasts to the DHCP server to collect the IP address, network configuration, and TFTP server information. After receiving the necessary information from DHCP, the PXE client connects to the TFTP server and downloads and loads the pxelinux.0 or similar network bootloader file from there. In the next step, the bootloader reads a config file like pxelinux.cfg/default and collects the menu and boot instructions. Finally, the kernel and initrd images are loaded via TFTP or HTTP, and the system boot is completed.

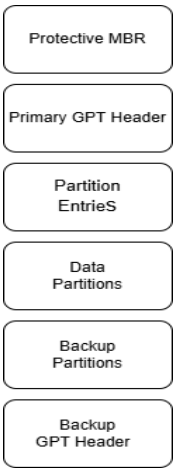


3. **GPT (GUID Partition Table):**

GPT is a modern disk partitioning scheme that uses GUID (Globally Unique Identifier) based partition tables. It overcomes the limitations of the older MBR. It supports large disk sizes (over 2 TB), can hold a much larger number of partitions (typically 128 or more), and has redundancy (primary + backup tables) against data loss.

2.1. General Layout of GPT and Explanation of Each Part

The typical layout of a GPT-based disk is usually like this:



Each part

1. **Protective MBR**

There is a "protective" MBR to prevent older MBR-only tools or OSes from thinking the entire disk is "empty" and overwriting it.

2. **Primary GPT Header**

In the GPT scheme, the main information is stored in the GPT header. It contains the GPT signature (“EFI PART”), the GPT version used, the unique GUID of the disk, and metadata such as where the partition entry array starts, how many partition entries there are, and the size of each entry. It also contains a CRC32 checksum, which is used to ensure the integrity of the header and partition table.

3. **Partition Entry Array**

The GPT scheme has a separate entry for each partition, which stores important metadata. Each entry contains a Partition Type GUID, which indicates the type of partition, and a Unique Partition GUID, which uniquely identifies it. It also contains the partition's Starting and Ending LBA, which specify the exact sector range of the disk in which the partition is located. Each entry also contains a UTF-16 encoded partition name. To hold multiple partitions, GPT typically uses a large partition entry array spanning multiple LBAs, such that a full block is allocated for 128 entries.

4. **Data Partitions (Original data area)**

This part contains the actual filesystem such as ext4, NTFS, FAT32, etc. The operating system, data, swap or other storage content is stored here according to the LBA range specified in the partition entry.

5. **Backup GPT Header and Backup Partition Entries**

A backup copy of the Primary GPT header and partition entries is kept at the very end of the disk. If the original GPT header or partition table at the beginning of the disk is damaged, the entire GPT structure can be easily restored using this backup, ensuring data security.

2.2. Main role of Partition Table:

The main role of the Partition Table is to make a disk organized, manageable, and suitable for multiple uses. First, the Partition Table is able to divide a physical disk into small logical parts, where each part can hold a different operating system, data, swap, or other specialized information. This makes it possible to easily manage multiple environments or tasks on the same disk. Second, it acts as a map of the disk for the OS and bootloader. The Partition Table indicates the start and end LBA of each partition, so the OS knows which block to read or write data from. Third, the Partition Table determines the type and purpose of the partition. The GPT scheme uses the Partition Type GUID to clearly specify which partition is an EFI System Partition, which is a Linux filesystem, or which is a Windows Basic Data. As a result, different operating systems and disk tools can easily understand the use of the partition. The GPT Partition Table also plays an important role in maintaining data security and consistency. It contains a CRC32 checksum, which helps verify whether the partition table has been corrupted. There is also a backup GPT table at the end of the disk, which allows for recovery if the primary table is damaged.

3. gdisk

gdisk (GPT fdisk) is a command-line tool used to create, modify, and maintain GPT partition tables. It is the modern equivalent of fdisk or cfdisk for MBR.

Working method:

When you run sudo gdisk /dev/sdX , gdisk first reads the disk's current GPT header and partition entries, and may offer to convert to GPT if there is an old MBR. It then presents an interactive menu, where n is used to create a new partition, d to delete, and p to view the table. All changes are initially in memory only; they are not actually written to the disk. Simply pressing w causes

gdisk to write the new GPT header and partition entries to the beginning and backup portions of the disk, and update the checksum.

What can usually be done with gdisk:

With gdisk, you can easily create new GPTs and change the structure of existing partitions. First, old MBR disks can easily be converted to GPT, which is essential for modern systems and large-sized disks. With gdisk, new partitions can be created according to the specified size, start-end LBA, and required partition type GUID. Similarly, it is possible to delete unnecessary partitions, resize them in some cases, and change the partition type. In addition, setting the label/name of each partition and changing its unique GUID can also be done from gdisk. gdisk can check the integrity of the GPT table. If there is any inconsistency between the Primary and Backup GPT headers or a CRC checksum mismatch, gdisk can detect them and repair them with the user's consent. As a result, it is possible to restore the damaged GPT structure. The gdisk is a flexible and powerful CLI tool.

3.3. If anyone wants to create a new Linux partition by partitioning the /dev/sdb disk with GPT, then first open gdisk by running sudo gdisk /dev/sdb. Then use the p command to view the current GPT table. If you press n to add a new partition, gdisk asks for the partition number, starting sector, ending sector or size, and partition type (default 8300). If the required size is entered, such as +20G, a 20GB partition will be created. User can verify whether the new entry has been added by issuing the p command again. If everything is correct, press w to actually write the changes to the disk and exit gdisk.

4. Protective MBR (PMBR)

Protective MBR is the MBR located at LBA 0 of a GPT disk, intended to prevent older MBR-based tools or OSes from accidentally overwriting the entire disk. It usually contains a single partition entry, with type 0xEE, starting at sector 1, and a size that is fixed across the entire disk.

Why GPT has Protective MBR:

The main focus of Protective MBR is to protect GPT disks from older MBR-only tools or OSes. Without it, an older fdisk or MBR-based system could think the disk is “empty” and write a new MBR table, destroying the GPT header and partition entries. Many older BIOS and utilities only read the MBR from the first sector, so when they see the 0xEE type of Protective MBR, they know the disk is specially protected and will not attempt to overwrite it. This creates a secure backward-compatible bridge between GPT and older systems.

Task 3 (References [17-25])

1. Verify the GPT schema of my Ubuntu machine

Step by step (Ubuntu)

1. **Opening the terminal and seeing the disk name:**

```
hasib@hasib-VMware-Virtual-Platform:~$ lsblk -o NAME,SIZE,TYPE,FSTYPE
NAME     SIZE TYPE FSTYPE
fd0        4K disk
loop0      4K loop squashfs
loop1   73.9M loop squashfs
loop2  245.1M loop squashfs
loop3    516M loop squashfs
loop4    576K loop squashfs
loop5   10.8M loop squashfs
loop6   49.3M loop squashfs
loop7   11.1M loop squashfs
loop8   91.7M loop squashfs
loop9   18.5M loop squashfs
sda       30G disk
├─sda1     1G part vfat
└─sda2  28.9G part ext4
sr0     95.3M rom  iso9660
sr1     1024M rom
hasib@hasib-VMware-Virtual-Platform:~$
```

Here I can see partitions like sda1, sda2, sr0, sr1 etc.

Since the main disk is NVME, its settings have been used everywhere.

2. **GPT or MBR is verified by running the following command, where gpt is confirmed:**

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo parted -l
[sudo] password for hasib:
Model: VMware, VMware Virtual S (scsi)
Disk /dev/sda: 32.2GB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start   End     Size    File system  Name  Flags
 1      1049kB  1128MB  1127MB  fat32              boot, esp
 2      1128MB  32.2GB  31.1GB  ext4
```

3. **Alternatively verify with gdisk**

   First gdisk is installed:

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt-get update
sudo apt-get install gdisk
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,30
9 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [215
kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21
.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata
 [9,416 B]
Get:10 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Package
s [2,140 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-e
n [486 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Compone
nts [212 B]
Get:13 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f M
```

Then when I run the following command, the output is (GPT fdisk (gdisk) version, Partition table scan, GPT: present, Total free space)

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo gdisk -l /dev/sda
GPT fdisk (gdisk) version 1.0.10

Partition table scan:
  MBR: protective
  BSD: not present
  APM: not present
  GPT: present

Found valid GPT with protective MBR; using GPT.
Disk /dev/sda: 62914560 sectors, 30.0 GiB
Model: VMware Virtual S
Sector size (logical/physical): 512/512 bytes
Disk identifier (GUID): C8D21E8A-68E0-4B1A-BAAF-5DCD23F7BE15
Partition table holds up to 128 entries
Main partition table begins at sector 2 and ends at sector 33
First usable sector is 2048, last usable sector is 62914526
Partitions will be aligned on 2048-sector boundaries
Total free space is 2015 sectors (1007.5 KiB)

Number  Start (sector)    End (sector)  Size       Code  Name
   1            2048         2203647   1.0 GiB     EF00
   2         2203648        62912511   28.9 GiB    8300
hasib@hasib-VMware-Virtual-Platform:~$
```

2. Use dd to dump the Protective MBR and GPT into a file

The dump is taken from the beginning of the disk to the Protective MBR, Primary GPT Header, and the first sector of the Partition Entry Array. This will include the first partition entry (and some other entries) - that is, the “up to first partition entry” condition will be met.

Disk /dev/sda. The dd command is powerful. If you mistakenly type of=/dev/sda, you can erase the entire disk.

2.1. Using dd step by step

In my case the disk /dev/sda is used. The dump is taken up to Protective MBR + GPT Header + first partition entry.

So I dump the first 3 sectors (3×512 = 1536 bytes):

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo dd if=/dev/sda of=~/gpt_dump.bin bs=
512 count=3
3+0 records in
3+0 records out
1536 bytes (1.5 kB, 1.5 KiB) copied, 0.000386103 s, 4.0 MB/s
hasib@hasib-VMware-Virtual-Platform:~$
```

if=/dev/sda indicates that the input source is your entire disk, which is read in read-only mode so that no changes are made. On the other hand, of=~/gpt_dump.bin is the output file, where the read data is saved in my home directory as gpt_dump.bin. This way, a raw binary copy of the entire disk can be safely saved.

Verify if the copy was successful:

```
hasib@hasib-VMware-Virtual-Platform:~$ ls -lh ~/gpt_dump.bin
-rw-r--r-- 1 root root 1.5K Nov 15 09:32 /home/hasib/gpt_dump.bin
hasib@hasib-VMware-Virtual-Platform:~$
```

The file size shows around 1.5K.

4. Installing GHex with GUI hex editor:

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt update
sudo apt install ghex -y
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
169 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libgtkhex-4-1
The following NEW packages will be installed:
  ghex libgtkhex-4-1
0 upgraded, 2 newly installed, 0 to remove and 169 not upgraded.
Need to get 437 kB of archives.
After this operation, 2,517 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 libgtkhex-4-1 amd64 46.0-1build1 [58.2 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 ghex amd64 46.0-1build1 [379 kB]
Fetched 437 kB in 1s (361 kB/s)
Selecting previously unselected package libgtkhex-4-1:amd64.
(Reading database ... 150067 files and directories currently installed.)
Preparing to unpack .../libgtkhex-4-1_46.0-1build1_amd64.deb ...
Unpacking libgtkhex-4-1:amd64 (46.0-1build1) ...
Selecting previously unselected package ghex.
Preparing to unpack .../ghex_46.0-1build1_amd64.deb ...
Unpacking ghex (46.0-1build1) ...
```

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo fdisk -l
Disk /dev/loop0: 4 KiB, 4096 bytes, 8 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/loop1: 11.13 MiB, 11673600 bytes, 22800 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/loop2: 73.91 MiB, 77504512 bytes, 151376 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/loop3: 245.13 MiB, 257036288 bytes, 502024 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/loop4: 91.69 MiB, 96141312 bytes, 187776 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

```
Disk /dev/fd0: 1.41 MiB, 1474560 bytes, 2880 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x90909090

Device     Boot      Start        End    Sectors  Size Id Type
/dev/fd0p1      2425393296 4850786591 2425393296  1.1T 90 unknown
/dev/fd0p2      2425393296 4850786591 2425393296  1.1T 90 unknown
/dev/fd0p3      2425393296 4850786591 2425393296  1.1T 90 unknown
/dev/fd0p4      2425393296 4850786591 2425393296  1.1T 90 unknown


Disk /dev/sda: 30 GiB, 32212254720 bytes, 62914560 sectors
Disk model: VMware Virtual S
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: C8D21E8A-68E0-4B1A-BAAF-5DCD23F7BE15

Device       Start      End  Sectors  Size Type
/dev/sda1     2048  2203647  2201600    1G EFI System
/dev/sda2  2203648 62912511 60708864 28.9G Linux filesystem


Disk /dev/loop8: 49.29 MiB, 51687424 bytes, 100952 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

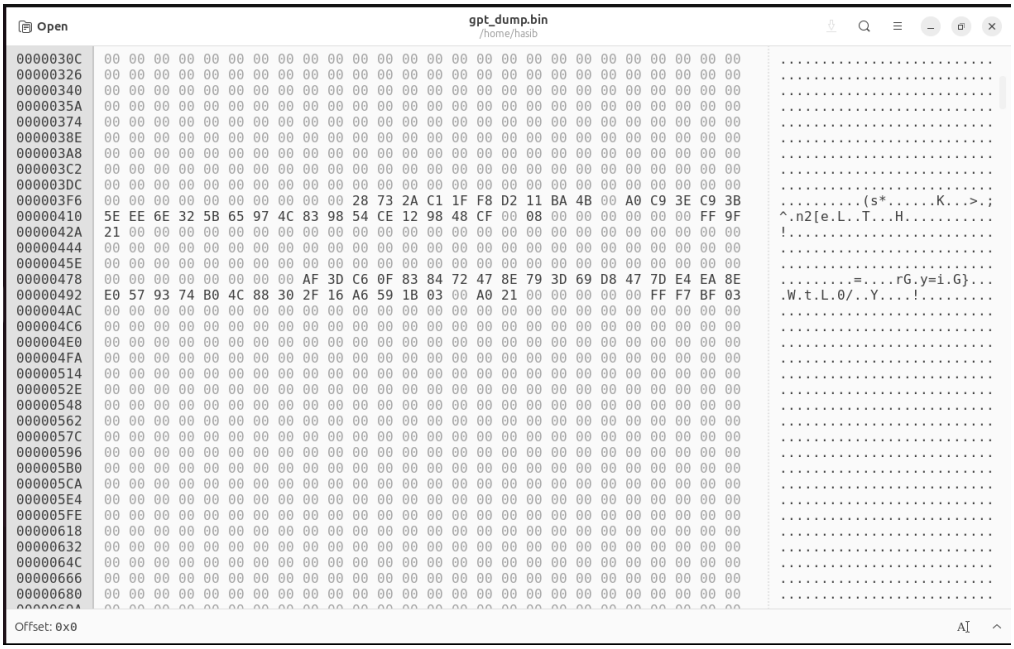
Show Apps

Creating a GPT dump:

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo dd if=/dev/sda of=~/gpt_dump.bin bs=512 count=34
34+0 records in
34+0 records out
17408 bytes (17 kB, 17 KiB) copied, 0.000438407 s, 39.7 MB/s
hasib@hasib-VMware-Virtual-Platform:~$
```

It is better to give ‘count=34’ because it takes a complete GPT backup (GPT header + partition entries).

Opening a file with GHex:



GPT dump analysis via terminal:

Basic analysis:

File size checked which is 17KB:

```
hasib@hasib-VMware-Virtual-Platform:~$ ls -lh ~/gpt_dump.bin
-rw-r--r-- 1 root root 17K Nov 15 09:44 /home/hasib/gpt_dump.bin
hasib@hasib-VMware-Virtual-Platform:~$
```

View hex dump:

```
hasib@hasib-VMware-Virtual-Platform:~$ hexdump -C ~/gpt_dump.bin | head -20
00000000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000001c0  02 00 ee ff ff ff 01 00  00 00 ff ff bf 03 00 00  |................|
000001d0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000001f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 55 aa  |..............U.|
00000200  45 46 49 20 50 41 52 54  00 00 01 00 5c 00 00 00  |EFI PART....\...|
00000210  45 d8 08 cb 00 00 00 00  01 00 00 00 00 00 00 00  |E...............|
00000220  ff ff bf 03 00 00 00 00  00 08 00 00 00 00 00 00  |................|
00000230  de ff bf 03 00 00 00 00  8a 1e d2 c8 e0 68 1a 4b  |.............h.K|
00000240  ba af 5d cd 23 f7 be 15  02 00 00 00 00 00 00 00  |..].#...........|
00000250  80 00 00 00 80 00 00 00  4e bb b6 3a 00 00 00 00  |........N..:....|
00000260  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000400  28 73 2a c1 1f f8 d2 11  ba 4b 00 a0 c9 3e c9 3b  |(s*......K...>.;|
00000410  5e ee 6e 32 5b 65 97 4c  83 98 54 ce 12 98 48 cf  |^.n2[e.L..T...H.|
00000420  00 08 00 00 00 00 00 00  ff 9f 21 00 00 00 00 00  |..........!.....|
00000430  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000480  af 3d c6 0f 83 84 72 47  8e 79 3d 69 d8 47 7d e4  |.=....rG.y=i.G}.|
```

Checking GPT signature:

```
hasib@hasib-VMware-Virtual-Platform:~$ hexdump -C ~/gpt_dump.bin | grep -A 2 -B 2 "EFI"
*
000001f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 55 aa  |..............U.|
00000200  45 46 49 20 50 41 52 54  00 00 01 00 5c 00 00 00  |EFI PART....\...|
00000210  45 d8 08 cb 00 00 00 00  01 00 00 00 00 00 00 00  |E...............|
00000220  ff ff bf 03 00 00 00 00  00 08 00 00 00 00 00 00  |................|
```

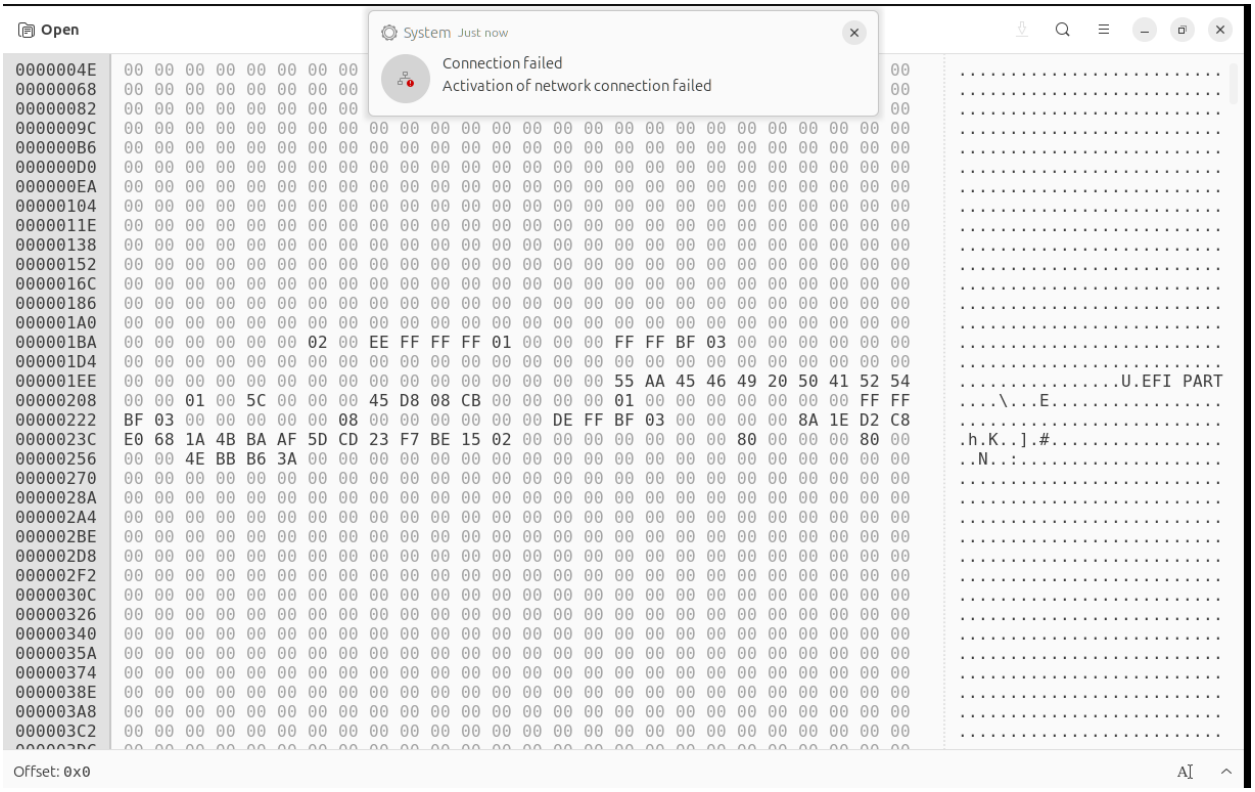
4. Annotate Protective MBR, GPT header, first partition entry

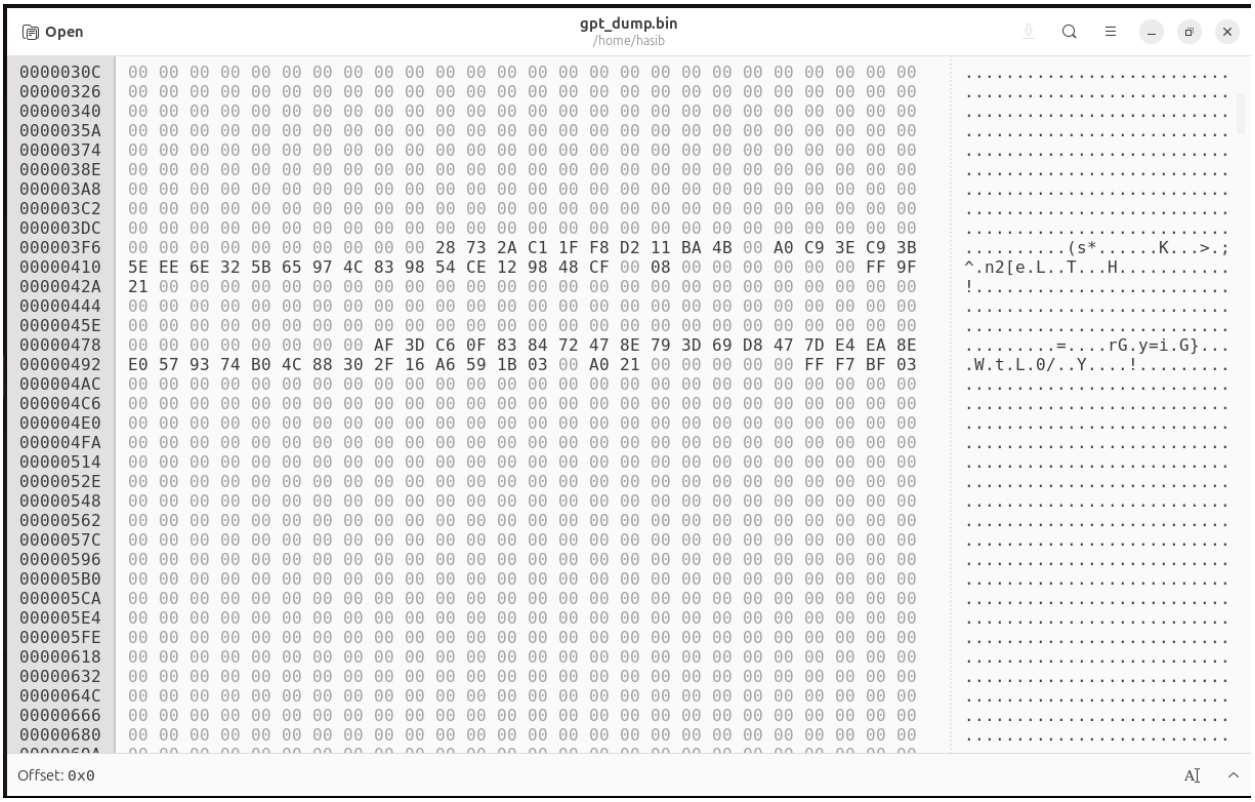
4.1. The GPT header shows:

- Partition entries start LBA = 2

If the sector size is 512 bytes: Byte offset = LBA × sector_size = 2 × 512 = 1024

Partition table entries start at the 1024th byte offset from the beginning of the disk (sector size 512 bytes).





4.2. Where to start the partition table for a 4Kn disk:

4Kn (4K native) Disc

Here each sector size = 4096 bytes. According to the GPT spec, header starts at LBA1, entries at LBA2, but each LBA is now 4096 bytes.

Partition entries start LBA = 2

Byte offset = 2 × 4096 = **8192**

If the server disk is 4Kn, the partition table entries start at the 8192nd byte offset from the beginning of the disk.

5. Name two differences between primary and logical partitions in MBR

This is not GPT, but the classic MBR partitioning system.

Difference 1:

Primary partition:

- A maximum of 4 primary partitions can be stored in the MBR.
- The entries are directly in the 4 slots of the MBR partition table.

Logical partition:

- For a logical partition, the primary slot is first used as an Extended partition.
- Inside the Extended partition, logical partitions are created one after another (forming a kind of chain inside the Extended).

Difference 2:

Primary partition:

- Stored directly in a 16-byte entry in the MBR partition table.
- Each primary partition can be separate, independently mounted, and bootable.

Logical partition:

- Their information is not stored in the original 4 entries of the MBR; rather, it is stored in additional smaller MBR-like records called EBRs inside the Extended partition.
- Logical partitions form a kind of linked-list-like structure, with each EBR pointing to the next logical partition.

References:

[1] Oracle Corporation, “Chapter 6: Virtual Networking,” *Oracle VM VirtualBox User Manual*. Available: https://www.virtualbox.org/manual/ch06.html.

[2] Canonical Ltd., “How to netboot the server installer on amd64,” *Ubuntu Server Documentation*, Oct. 2025. Available: https://documentation.ubuntu.com/server/how-to/installation/how-to-netboot-the-server-installer-on-amd64/.

[3] Canonical Ltd., “Netboot the server installer via UEFI PXE on ARM (aarch64, arm64) and x86_64 (amd64),” *Ubuntu Server Documentation*, Oct. 2025. Available: https://documentation.ubuntu.com/server/how-to/installation/netboot-the-server-installer-via-uefi-pxe-on-arm-aarch64-arm64-and-x86-64-amd64/.

[4] G. Fonteneau, “Ubuntu 24.04 - Deploy Ubuntu 24.04 Server through PXE (BIOS & UEFI),” *C-nergy Tech Blog*, Feb. 2025. Available: https://c-nergy.be/blog/?p=20027.

[5] L. Uppeng, “Network boot Ubuntu Server ISO via Preboot Execution Environment (PXE),” *Luppeng Blog*, Jul. 2024. Available: https://luppeng.wordpress.com/2024/07/02/network-boot-ubuntu-server-iso-via-preboot-execution-environment-pxe/.

[6] “dnsmasq(8) - a lightweight DHCP and caching DNS server,” *Debian/Ubuntu Manpages*. Available: https://manpages.debian.org/dnsmasq-base/dnsmasq.8.en.html.

[7] Unified Extensible Firmware Interface Forum, “Secure boot and driver signing,” in *UEFI Specification Version 2.10*, ch. 32. Available: https://uefi.org/specs/UEFI/2.10/32_Secure_Boot_and_Driver_Signing.html.

[8] Microsoft, “Secure boot,” *Windows Hardware Dev Center*. Available: https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/oem-secure-boot.

[9] Ivanti, “About Legacy BIOS PXE boot or UEFI PXE boot—What’s the difference, why and when it matters,” *Ivanti Community Documentation*, 2022. Available: https://forums.ivanti.com/s/article/About-Legacy-BIOS-PXE-boot-or-UEFI-PXE-boot-What-s-the-difference-why-and-when-it-matters.

[10] Intel Corporation, “Preboot Execution Environment (PXE) Specification, Version 2.1,” 1999. Available: https://docs.smoe.org/misc/pxe-2.1-spec.pdf.

[11] Unified Extensible Firmware Interface Forum, “GUID partition table (GPT) disk layout,” in *UEFI Specification Version 2.10*, ch. 5. Available: https://uefi.org/specs/UEFI/2.10/05_GUID_Partition_Table_Format.html.

[12] “GUID Partition Table,” *Wikipedia*. Available: https://en.wikipedia.org/wiki/GUID_Partition_Table.

[13] R. M. Baldwin, “Understanding the Protective MBR in GPT-partitioned disks. Part 1,” *Lernme.com*, 2024. [Online]. Available: https://lernme.com/1-2/.

[14] “gdisk(8) - Interactive GUID partition table (GPT) manipulator,” *Linux man page*, Linux.die.net. [Online]. Available: https://linux.die.net/man/8/gdisk.

[15] M. Kerrisk, “fdisk(8) - partition table manipulator for Linux,” *man7.org Linux manual pages*, 2024. Available: https://www.man7.org/linux/man-pages/man8/fdisk.8.html.

[16] “Legacy BIOS PXE boot vs. UEFI: What you need to know,” *Cleverence Tech Blog*, 2025. Available: https://www.cleverence.com/articles/tech-blog/legacy-bios-pxe-boot-vs-uefi-what-you-need-to-know/.

[17] Unified Extensible Firmware Interface Forum,“GUID Partition Table (GPT) Disk Layout,” *UEFI Specification Version 2.10*, Section 5, 2022. Available: https://uefi.org/specs/UEFI/2.10/05_GUID_Partition_Table_Format.html

[18] R. Baldwin, “Understanding the Protective MBR in GPT-partitioned disks (Part 1),” *Lernme Technical Blog*, 2024. Available: https://lernme.com/1-2/

[19] R. Baldwin,“Understanding the Protective MBR in GPT-partitioned disks (Part 2),” *Lernme Technical Blog*, 2024. Available: https://lernme.com/1-3/

[20] “gdisk(8) - Interactive GUID partition table manipulator,” Linux Man-Pages, Linux.die.net. Available: https://linux.die.net/man/8/gdisk

[21] M. Kerrisk, “fdisk(8) - Partition table manipulator for Linux,” *man7.org Linux Manual Pages*, 2024. Available: https://man7.org/linux/man-pages/man8/fdisk.8.html

[22] Canonical Ltd., “Examine disk partitioning, check MBR/GPT, and use dd safely,” *Ubuntu Documentation*, 2024. Available: https://ubuntu.com/server/docs

[23] Wikipedia, “GUID Partition Table,” 2025. Available: https://en.wikipedia.org/wiki/GUID_Partition_Table

[24] Intel Corporation,“Extensible Firmware Interface - GPT and Partition Tables,” *EFI Specification*, v1.10, 2001. Available: https://uefi.org/specifications

[25] GNU dd manual, “Copying and converting data: dd,” *GNU Coreutils*, Free Software Foundation. Available: https://www.gnu.org/software/coreutils/manual/html_node/dd-invocation.html